Code Patterns

Use this only for project-specific conventions. Nothing has been implemented yet, so most sections record the conventions the design commits to rather than patterns observed in existing code. Update a section from the real code once it exists, and delete conventions the team decides against.

Architecture

- Primary pattern: layered by trust boundary, not feature folders. The trusted host (GUI, job controller, acceptance engine, artifact store, provider adapter) is separate from the untrusted worker code that runs inside the disposable guest.
- Route the flow through named responsibilities: `app/api` (typed HTTP routes) → `app/services` (job controller, provider adapter, artifact/export) → `app/domain` (contracts, candidate schema, phases/outcomes, acceptance rules) → `app/db` (parameterized SQLite) and the artifact store. Workers under `workers/` never touch the database.
- Keep the provider adapter behind an interface — `request_candidate(context, route, limits) -> validated candidate | typed failure` — so the model/endpoint is replaceable and the product boundary stays in one module.
- The guest broker accepts a fixed operation list (analyze, assemble, compile, apply, test, verify) with typed manifests and artifact IDs. Never a shell string, never an arbitrary path.
- Reuse an existing module before creating a new abstraction. This project has few moving parts on purpose.

Data And State

- **Identity:** SHA-256 for input, contract, manifest, and candidate identity. Hashes establish identity, not correctness or authenticity — never infer trust from a matching hash.
- **Phases vs outcomes:** they are separate axes. Phases describe where work is; outcomes are the six result labels. A missing or not-implemented check is recorded as `not_run`, never `passed`.
- **Approval binding:** an approval binds the original hash, contract revision, test-manifest hash, patch policy, environment profile, and verification scope. Any edit to a bound input invalidates the approval and forces a new run. Earlier runs are preserved, never overwritten.
- **Artifact writes:** write to a private temp file, hash and validate it, rename atomically into the job directory, then register metadata in a transaction. Reconcile interrupted writes on startup — an unregistered file can never satisfy an obligation.
- **Idempotency:** runs start with an idempotency key so a double click cannot create duplicate charged attempts. Cancellation is a persisted request followed by termination; record `cancelled` only after termination is confirmed, otherwise `error`/`termination_unconfirmed` and lock the guest until it is reset.
- **Transactions:** keep them short, enable foreign keys, set a busy timeout. One writer at a time is expected.
- **Leases:** one controller leases one active job; routes run sequentially by default. Heartbeats identify a lost worker.
- Data fetching in the UI: server-rendered pages plus small JSON status fetches polling persisted events every 2 s. Never synthesise a progress percentage.

- Client state stays in SQLite and the artifact store; the browser holds only what it is currently displaying.

Errors And Validation

- Validate at the boundary: Pydantic models for every request, typed test manifests, uploaded content checked by content rather than extension, streamed size limits and quotas.
- The request model cannot express arbitrary host commands, shell scripts, or unrestricted filesystem paths. Tool arguments are built from validated values and passed as argument arrays.
- Return user-safe errors: a correlation ID on every response, and no shell traceback or secret-bearing provider body reaches the browser.
- Use the agreed status codes: 422 invalid structured input, 413 size limit, 401/403 session or origin, 404 unavailable object, 409 stale approval or invalid transition.
- Job-level unsupported capability is expressed in job evidence, not by returning HTTP success as if it meant repair success.
- Do not swallow errors. An interrupted or failed stage preserves the evidence that was already produced.
- Typed failures carry a `reason_code`; `error` and `cancelled` stay distinct from `rejected` and `inconclusive`.
- Never let a failure path assign `passed` to a check.

Naming

- Files and modules: `snake_case` (`job_controller.py`, `acceptance_rules.py`).
- Classes and Pydantic models: `PascalCase`.
- Functions and variables: `snake_case`.
- Constants and environment variables: `UPPER_SNAKE_CASE`, prefixed `BINARYREPAIR_` for app settings.
- Database objects: `snake_case` tables and columns, matching the Tech Design's field names so the schema and the domain models stay readable together.
- Route and outcome labels keep their exact published spelling (`success_within_scope`, `not_run`, `roundtrip`) wherever they are persisted, compared, or shown — these are contract values, not display strings to reword.
- Test files: `tests/<group>/test_<subject>.py`, named after the behaviour under test rather than the module it happens to call.

API And UI Patterns

- Three views only: setup/review, repair comparison, evidence/export. Templates are server-rendered; JavaScript fetches small status updates and nothing else.
- Status is always shown with text or an icon as well as colour. Completed, pending, unavailable, and diagnostic results must stay visually distinct.
- Limitations belong next to the result they qualify, with technical detail expandable behind the main outcome.
- Route cards must not imply that a smaller patch is automatically safer.
- Completed evidence survives a later failing stage.
- Render untrusted output through template escaping and DOM `textContent`. Generated evidence HTML is rendered through trusted templates; worker-supplied HTML is never shown as active

content.
- Force downloads for binaries and raw artifacts, with attachment disposition and a safe filename built from registered relative paths.
- Every success label comes from the acceptance engine. Templates render outcomes; they never compute them.

AI Tool Patterns

The product integrates an AI provider, so these conventions apply to that integration.

- Keep the tool surface minimal: the model proposes a typed candidate and nothing else. It cannot execute, write, delete, request permissions, or reach the network.
- The server supplies allowed locations, interfaces, and limits. A candidate can never expand them.
- Validate model input and structured output locally, including hashes, integer ranges, target membership, string sizes, route-specific fields, and forbidden extra fields. Provider JSON mode is a transport convenience, not schema validity.
- Treat binary-derived excerpts, decompiler output, generated C and assembly, guest output, retrieved pages, and tool output as untrusted data. Embedded instructions never change scope, permissions, or tool availability.
- Require approval for destructive, external-network, credential-bearing, and production actions; default-deny external transfer of client material unless an explicit client authorization covers it.
- Log prompt hashes, model/settings identifiers, status, and usage; never log prompt bodies containing confidential content, keys, or session tokens. Unknown usage is labelled unavailable, not zero.
- Never fabricate a model response or a positive verification, and never let a deterministic template baseline be presented as a successful AI-assisted route.